

DOCUMENT #2 OF 10 — TECHNICAL SPECIFICATION

Technical Requirement Document (TRD)

Bespoke Stack: React 19, Vite, TypeScript, Supabase Backend, and LLM APIs

Architecture Style

BaaS + Edge SPA +
Serverless AI

Frontend / Build

React 19 + TypeScript + Vite
Supabase Cloud +
PostgreSQL (RLS)

Backend & Database

AI Integration

Google Gemini / Claude API
+ Web Speech

1. System Architectural Overview

The application utilizes a lightweight, modern Serverless / Backend-as-a-Service (BaaS) architecture. The client-side SPA (React 19 + TypeScript + Vite) connects directly to Supabase via its high-performance JavaScript SDK for auto-generated REST/GraphQL queries, real-time subscriptions, secure auth, and storage. AI parsing and evaluation logic are handled via serverless Edge Functions connecting to Google Gemini / Claude APIs.

```
[ User / Recruiter Browser (SPA) ]
├── [ Frontend Layer ]: React 19 + TypeScript + Vite + Custom Glassmorphism CSS
│   ├── React Router (SPA Navigation)
│   ├── Web Speech API (Live Voice-to-Text Input)
│   └── Leaflet.js (Interactive Geolocation / Facility Tracking)
├── [ Authentication & Storage Layer ] (Supabase Client SDK)
│   ├── Supabase Auth (JWT, OAuth: Google/Apple, Session Handling)
│   └── Supabase Storage Buckets (Encrypted Resume & Document Uploads)
├── [ Data Layer & Row Level Security ]
│   └── Supabase Hosted PostgreSQL 16 (Auto-generated APIs + RLS Isolation)
└── [ AI & LLM Inference Layer ]
    ├── Supabase Edge Functions / Direct Secure Proxy
    │   ├── Google Gemini API / Anthropic Claude API (NER & Parsing, Match Scoring)
    │   └── Claude-style Conversational UI Engine
```

2. Complete Specified Technology Stack

1. Frontend Architecture

- **Core Framework:** React 19 with TypeScript
- **Build Tool:** Vite (Fast HMR, optimized rollups)
- **Routing:** React Router (Client-side SPA)
- **Styling:** Custom Vanilla CSS (CSS Variables, Modern Grid/Flexbox, Glassmorphism UI)
- **Mapping:** Leaflet.js (Candidate location & office tracking)

3. Authentication & Security

- **Provider:** Supabase Auth
- **Auth Features:** Email/Password, Magic Link, OAuth (Google/Apple)
- **Access Control:** PostgreSQL RLS Policies per tenant & user
- **Tokens:** Secure HTTP/JWT automated session tokens

2. Backend & Data Layer

- **BaaS Platform:** Supabase (Auto REST & GraphQL APIs)
- **Database:** PostgreSQL with Row Level Security (RLS)
- **File Storage:** Supabase Storage Buckets (PDF/DOCX)
- **Edge Runtime:** Supabase Edge Functions (Deno/TypeScript)

4. AI Models & Real-Time APIs

- **LLM Providers:** Google Gemini API / Anthropic Claude API
- **UI Pattern:** "Claude-style" conversational streaming interfaces
- **Voice API:** Browser Native Web Speech API (Voice-to-Text)
- **Version Control & CI:** Git + GitHub Repository

3. AI Pipeline & Chat Interface Flow

Phase	Technical Operation	Engine / Service Used
1. File Upload & Storage	Resumes/Scans uploaded via client → securely saved into private buckets with signed URLs.	Supabase Storage (Signed URLs)
2. Text & Voice Ingestion	Voice queries transcribed client-side; document text sent to serverless AI endpoint.	Web Speech API + Supabase Edge Functions
3. AI Reasoning & Match	Contextual LLM prompt extracts entities, compares against Job criteria, and computes 0–100 match.	Google Gemini / Claude API
4. Conversational Feedback	Streaming assistant output rendered in real-time "Claude-style" chat bubbles.	SSE (Server-Sent Events) + Custom React Hooks
5. Persistent Results	Candidate scores, match logs, and chat transcripts written to PostgreSQL with strict RLS.	Supabase PostgreSQL + RLS

4. Database & Row Level Security (RLS) Model

ROW LEVEL SECURITY (RLS) ENFORCEMENT

All relational tables in PostgreSQL have RLS enabled by default. Recruiters can only access candidates and jobs belonging to their authenticated workspace, while candidates can only read their own submitted documents and scores:

```
-- Example RLS Policy for Resumes Table
ALTER TABLE resumes ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can only access their own organization resumes"
ON resumes FOR ALL
USING (auth.uid() = user_id OR auth.jwt() ->> 'role' = 'super_admin');
```

5. Engineering Rules & Production Standards

- **TypeScript Strict Mode:** All frontend React components and utility hooks must have zero any types.
- **Pure CSS Glassmorphism:** Leverage native CSS variables (--glass-bg, --glass-border, backdrop-filter: blur(12px)) without heavy external CSS libraries.
- **Edge Function Security:** Never expose raw Gemini/Claude API keys on the frontend; all AI scoring requests must route through authenticated Supabase Edge Functions.

- **Optimistic UI & Cache Invalidation:** Use React 19 hooks and Supabase real-time channels for instant UI updates.

6. Deployment & Hosting Strategy

Frontend Edge Hosting

Hosted on **Vercel**, **Netlify**, or **Firebase Hosting** with global CDN edge caching and automated Git push-to-deploy pipelines.

Backend & Storage Cloud

Fully managed serverless infrastructure hosted on **Supabase Cloud** with automated daily backups, managed PostgreSQL, and global S3-compatible storage.